

Pipeline: Procesamiento de Reservas

Procesa las solicitudes de reserva realizadas por los usuarios, validando los datos ingresados, verificando la disponibilidad de recursos y registrando el turno correspondiente en el sistema.

PASO 1 — ORIGEN DE LOS DATOS

¿Qué datos?	Solicitud de reserva en formato JSON compuesta por {email, fecha, hora, tipo_partido, id_cancha, cupos_disponibles, [id_equipamiento, cantidad]}
¿Desde dónde?	Formulario de reserva de la aplicación web, enviado al endpoint de creación de reservas
¿Con qué frecuencia llegan?	Ante cada solicitud de reserva realizada por Usuario Cliente o por un Usuario Auxiliar mediante una reserva manual
¿Volumen estimado?	Medio a alto, debido a que corresponde al flujo principal del negocio y depende de la cantidad de usuarios, complejos y canchas gestionadas.



PASO 2 — DISPARADOR / TRIGGER

¿Cuándo se ejecuta?	Ante un evento de negocio: la creación de una solicitud de reserva. La frecuencia es de baja demanda, dependiendo de la cantidad de complejos y usuarios gestionados. Se trata de un procesamiento evento-driven.
¿Quién lo dispara?	Request HTTP POST enviado desde la aplicación web por un Usuario Cliente/Auxiliar hacia Turnos API



PASO 3 — PROCESAMIENTO

¿Qué componente lo ejecuta?	El procesamiento lo ejecuta Turnos API mediante el Componente de Turnos que utiliza Next.js backend.
Paso 1 — Recepción de solicitud	El componente recibe la solicitud de reserva enviada desde la interfaz web y extrae los datos necesarios para su procesamiento.
Paso 2 — Validación de Datos	Verifica que la información recibida sea válida y que todos los campos obligatorios se encuentren completos. También valida que el Usuario Cliente no se encuentre penalizado.
Paso 3 — Verificación de disponibilidad	Consulta las reservas existentes para determinar si el turno solicitado está disponible.
Paso 4 — Verificación de equipamiento	Si se solicitó equipamiento, verifica que haya stock suficiente para satisfacer la solicitud.
Paso 5 — Asignación de estado inicial	El sistema registra la reserva. Las reservas realizadas por un Usuario Cliente se crean en estado pendiente. Las reservas manuales registradas por un Usuario Auxiliar se crean en estado confirmada, ya que el pago se realiza presencialmente en el complejo. Si la reserva corresponde a un partido abierto, el campo cuposDisponibles queda disponible para el Pipeline de Búsqueda de Equipo.
Paso 6 — Generación de Eventos	El sistema genera los eventos necesarios para que los Pipelines de Procesamiento de Pagos, Búsqueda de Equipo y Notificaciones continúen su procesamiento según corresponda.
¿Hay lógica de error?	Si los datos ingresados son inválidos, la cancha no se encuentra disponible, el usuario no existe o el equipamiento solicitado supera el stock disponible, el procesamiento se interrumpe, la reserva no se registra y se devuelve un mensaje indicando la causa del error.

	Los errores relacionados con el pago no son gestionados por este pipeline, sino por el Pipeline de Procesamiento de Pagos.
--	--



PASO 4 — SALIDA / DESTINO

¿Qué se produce?	Una reserva registrada en el sistema con estado pendiente, asociada a un Usuario Cliente y a una Cancha. En caso de corresponder, se genera una solicitud para el Pipeline de Procesamiento de Pagos. Si el usuario solicita un partido abierto, también se genera una solicitud para el Pipeline de Búsqueda de Equipo.
¿Dónde se guarda?	Se guarda en la tabla Reserva en la base de datos, incluye el equipamiento solicitado.
¿Quién consume el resultado?	El frontend de la aplicación para informar el resultado al usuario, el Pipeline de Procesamiento de Pagos cuando la reserva requiere pago anticipado, el Pipeline de Búsqueda de Equipo cuando se solicita un partido abierto, el Pipeline de Notificaciones para comunicar eventos relacionados con la reserva y el Pipeline de Estadísticas para el análisis posterior de la actividad del sistema.



PASO 5 — DECISIÓN ARQUITECTÓNICA

¿Por qué este enfoque?	Se optó por un procesamiento orientado a eventos, ya que las reservas deben gestionarse en tiempo real para garantizar que la disponibilidad de las canchas se encuentre siempre actualizada y evitar conflictos entre solicitudes simultáneas. Además, el Pipeline de Reservas se mantiene separado de pagos, notificaciones, búsqueda de equipo y estadísticas para conservar responsabilidades claras.
Trade-offs aceptados	La separación en múltiples pipelines incrementa la cantidad de componentes y la complejidad de coordinación entre ellos. Asimismo, este enfoque requiere un mayor manejo de errores y eventos entre procesos. Sin embargo, mejora la modularidad, reduce el acoplamiento entre funcionalidades y facilita futuras modificaciones y ampliaciones del sistema. Se asume que las reservas registradas manualmente por el Usuario Auxiliar ya fueron abonadas presencialmente, por lo que se crean directamente en estado confirmada. Este enfoque simplifica el procesamiento y evita que dichas reservas pasen por el Pipeline de Pagos. Sin embargo, depende de que el registro realizado por el auxiliar sea correcto, ya que el sistema no verifica automáticamente que el pago en efectivo haya sido efectivamente realizado. Debido a la arquitectura monolítica modular adoptada, la interacción entre pipelines se implementa mediante llamadas internas entre componentes del backend. Conceptualmente, estas interacciones se modelan como eventos de negocio.

⇒ Hace referencia ADR-001, ADR-003